Backend Code Walkthrough: 4-Person Team Guide

Structured breakdown, speaking scripts, and Q&A; points for backend.cpp

Member	Assigned Section & Lines	Core Focus / DSA Concepts
Person 1	`Lines 1–46 (Pigeon & Pigeonhole)`	Domain entities, dynamic vector container, bucket encapsulation.
Person 2	`Lines 48–118 (PigeonholeTable)`	Hash table mapping, O(1) modulo hash, theorem calculation ⌈n/k⌉.
Person 3	`Lines 120–195 (Service & Serialization)`	Simulation bounds clamping, URL parsing, JSON formatting & CORS.
Person 4	`Lines 197–257 (Winsock2 Networking)`	Socket setup, TCP binding/listening, event loop, HTTP request dispatch.

■ Person 1: Domain Entities & Bucket Data Structures (Lines 1–46)

Code Scope: Header inclusions, struct Pigeon, and class Pigeonhole

- **Item Node (struct Pigeon):** Encapsulates individual objects with a unique identifier id.
- **Bucket Container (class Pigeonhole):** Uses dynamic array std::vector<Pigeon> occupants to store placed items.
- **Methods:** addPigeon() (inserts item), getCount() (returns load), hasCollision() (returns true when count > 1).

■■ ***Speaking Script:***
"I am responsible for the foundational data models in our backend. We modeled the Pigeonhole problem using Object-Oriented principles. 'Pigeon' represents an individual item node with an ID. 'Pigeonhole' represents each container bucket, utilizing a dynamic array (std::vector) to store its occupants. It provides helper methods such as addPigeon() and hasCollision(), which instantly detects if more than one object lands in the same slot."

■ Person 2: Hash Table & Mathematical Engine (Lines 48–118)

Code Scope: class PigeonholeTable (Hash function, distribution, and mathematical theorems)

- **Hash Table Model:** Maintains an array of buckets vector<Pigeonhole> holes and an assignment map vector<int> assignments.
- **Hash Mapping:** hashFunction(i) = i % numHoles runs in **O(1)** time complexity.
- **Distribution:** distribute() iterates through all *n* objects and inserts them into mapped buckets in **O(n)** time.
- **Pigeonhole Guarantee:** Implements (totalObjects + numHoles - 1) / numHoles to compute ⌈n / k⌉.

■■ ***Speaking Script:***
"I handle the core computational structure, PigeonholeTable. It functions as a Hash Table with separate chaining. The hashFunction maps each item using modulo hashing in O(1) time. The distribute() method places all n objects into their corresponding buckets in O(n) time. It also computes the mathematical guarantee ceil(n/k) using integer ceiling division, mathematically proving the minimum density in the maximally occupied bucket."

■ Person 3: Service Layer, Query Parser & JSON Serializer (Lines 120–195)

Code Scope: SimulationResult, runSimulation(), parseQueryParameters(), and serializeToJson()

- **DTO & Clamping:** SimulationResult packages output; clampValue() ensures valid safe ranges (objects: 1–30, boxes: 1–12).
- **URL Query Parser:** Uses std::stringstream and string splitting to parse key-value pairs into a std::map.
- **JSON & Response:** Serializes distribution arrays into JSON and attaches HTTP headers (including CORS Access-Control-Allow-Origin: *).

■■ ***Speaking Script:***
"My part connects the core data structures with the web API. runSimulation() validates and clamps user inputs before executing the table distribution. I implemented parseQueryParameters() using stringstream to extract URL parameters like objects and boxes. Finally, serializeToJson() converts the simulation results into JSON, and sendHttpResponse() wraps it with standard HTTP/1.1 headers and CORS support so our frontend can display it seamlessly."

■ Person 4: Winsock Networking & Server Main Loop (Lines 197–257)

Code Scope: main() entry point, Winsock2 socket setup, listening loop, and client connection management

- **Winsock Setup:** Initializes Windows Sockets with WSAStartup(2.2) and creates a TCP socket (AF_INET, SOCK_STREAM).
- **Binding & Listening:** Binds socket to localhost on port 8080 and starts listening via listen(server, SOMAXCONN).
- **Event Loop & Routing:** Listens in a while(true) loop, accepts client connections via accept(), reads buffer via recv(), routes GET /api/simulate, and safely closes client sockets.

■■ ***Speaking Script:***
"I handle the networking infrastructure and runtime server loop. We utilized the native Windows Sockets (Winsock2) library to implement a high-performance HTTP server without external third-party dependencies. In main(), we initialize Winsock, bind to port 8080, and enter an accept loop. When a request arrives, the server checks the route, executes Person 3's simulation pipeline, returns the HTTP response, and gracefully closes the connection."

■ Viva & Presentation Q&A; Cheat Sheet

Question	Target Person & Short Answer
Why use std::vector inside Pigeonhole?	**Person 1:** To support dynamic separate chaining without fixed size limits.
What is the time complexity of the distribution?	**Person 2:** *O(n)* overall because hashing each item takes *O(1)* time.
Why is CORS header needed in HTTP response?	**Person 3:** To allow the frontend running on port 3000 to fetch from port 8080.
Why use SO_REUSEADDR on the socket?	**Person 4:** To allow immediate rebinding to port 8080 without port lock errors.